

An Analysis of Policy gradient method

Policy gradient method

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi}(s, a)]$$

Policy gradient method -- Part 1

Motivation Standard policy gradient updates $\theta_{i+1} = \theta_i + \alpha \nabla_{\theta} J(\theta_i)$ solve a constrained optimization problem: $\max_{\theta} \theta_{i+1} J(\theta_i) + (\theta_{i+1} - \theta_i) T \nabla_{\theta} J(\theta_i) \leq \alpha \cdot \nabla_{\theta} J(\theta_i)$

Policy gradient method -- Part 2

While the objective (linearized improvement) is geometrically meaningful, the Euclidean constraint $\theta_{i+1} - \theta_i$ introduces coordinate dependence. To address this, the natural policy gradient replaces the Euclidean constraint with a Kullback–Leibler divergence (KL) constraint: $\max_{\theta} \theta_{i+1} J(\theta_i) + (\theta_{i+1} - \theta_i) T \nabla_{\theta} J(\theta_i) D^{-1} KL(\pi_{\theta_{i+1}} \parallel \pi_{\theta_i}) \leq \alpha$ where the KL divergence between two policies is averaged over the state distribution under policy π_{θ_i} . That is, $D^{-1} KL(\pi_{\theta_{i+1}} \parallel \pi_{\theta_i}) := \mathbb{E}_{s \sim \pi_{\theta_i}} [D KL(\pi_{\theta_{i+1}}(\cdot | s) \parallel \pi_{\theta_i}(\cdot | s))]$ This ensures updates are invariant to invertible affine parameter transformations.

Policy gradient method -- Part 3

Policy gradient methods are a class of reinforcement learning algorithms and a sub-class of policy optimization methods. Unlike value-based methods which learn a value function to derive a policy, policy optimization methods directly learn a policy function π that selects actions without consulting a value function. For policy gradient to apply, the policy function π_{θ} is parameterized by a differentiable parameter θ .

Policy gradient method -- Part 4

Variance reduction REINFORCE is an on-policy algorithm, meaning that the trajectories used for the update must be sampled from the current policy π_{θ} . This can lead to high variance in the updates, as the returns $R(\tau)$ can vary significantly between trajectories. Many variants of REINFORCE have been introduced, under the title of variance reduction.

Policy gradient method -- Part 5

$\nabla_{\theta} J(\theta) = \mathbb{E}_{(s,a) \sim \pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a | s) \cdot A^{\pi_{\theta}}(s, a)] = \nabla_{\theta} L(\theta, \theta_t)$ However, when $\theta \neq \theta_i$, this is not necessarily true. Thus it is a "surrogate" of the real objective. As with natural policy gradient, for small policy updates, TRPO approximates the surrogate advantage and KL divergence using Taylor expansions around θ_t : $L(\theta, \theta_i) \approx g(\theta - \theta_i)$, $D^{-1} KL(\pi_{\theta} \parallel \pi_{\theta_i}) \approx \frac{1}{2} (\theta - \theta_i)^T H(\theta - \theta_i)$, where $g = \nabla_{\theta} L(\theta, \theta_i)$ and $H = \nabla^2_{\theta} L(\theta, \theta_i)$.

Policy gradient method -- Part 6

$\gamma \sum_{t=0}^{\tau-1} \gamma^t R_t - b(S_t)$: used by the REINFORCE with baseline algorithm.

Policy gradient method -- Part 7

Overview In policy-based RL, the actor is a parameterized policy function π_{θ} , where θ are the parameters of the actor. The actor takes as argument the state of the environment s and produces a probability distribution $\pi_{\theta}(\cdot | s)$. If the action space is discrete, then $\sum_a \pi_{\theta}(a | s) = 1$. If the action space is continuous, then $\int \pi_{\theta}(a | s) da = 1$. The goal of policy optimization is to find some θ that maximizes the expected episodic reward $J(\theta) = \mathbb{E}_{\pi_{\theta}} [\sum_{t=0}^T \gamma^t R_t | S_0 = s_0]$ where γ is the discount factor, R_t is the reward at step t , s_0 is the starting state,

An Analysis of Policy gradient method

Policy gradient method

and T is the time-horizon (which can be infinite). The policy gradient is defined as $\nabla_{\theta} J(\theta)$. Different policy gradient methods stochastically estimate the policy gradient in different ways. The goal of any policy gradient method is to iteratively maximize $J(\theta)$ by gradient ascent. Since the key part of any policy gradient method is the stochastic estimation of the policy gradient, they are also studied under the title of "Monte Carlo gradient estimation".

Policy gradient method -- Part 8

A further improvement is proximal policy optimization (PPO), which avoids even computing $F(\theta)$ and $F(\theta) - 1$ via a first-order approximation using clipped probability ratios. Specifically, instead of maximizing the surrogate advantage $\max_{\theta} L(\theta, \theta_t) = \mathbb{E}_{s, a \sim \pi_{\theta_t}} [L(\theta, \theta_t)(s, a)]$, PPO directly inserts the constraint into the surrogate advantage: $\max_{\theta} \mathbb{E}_{s, a \sim \pi_{\theta_t}} [L(\theta, \theta_t)(s, a) \cdot \min(1, \frac{\pi_{\theta}(a|s)}{\pi_{\theta_t}(a|s)})]$ under a KL divergence constraint, it directly inserts the constraint into the surrogate advantage: $\max_{\theta} \mathbb{E}_{s, a \sim \pi_{\theta_t}} [L(\theta, \theta_t)(s, a) \cdot \min(1, \frac{\pi_{\theta}(a|s)}{\pi_{\theta_t}(a|s)})]$ and PPO maximizes the surrogate advantage by stochastic gradient descent, as usual. In words, gradient-ascenting the new surrogate advantage function means that, at some state s, a , if the advantage is positive: $A^{\pi_{\theta_t}}(s, a) > 0$, then the gradient should direct θ towards the direction that increases the probability of performing action a under the state s . However, as soon as θ has changed so much that $\pi_{\theta}(a|s) \geq (1 + \epsilon) \pi_{\theta_t}(a|s)$, then the gradient should stop pointing it in that direction. And similarly if $A^{\pi_{\theta_t}}(s, a) < 0$. Thus, PPO avoids pushing the parameter update too hard, and avoids changing the policy too much. To be more precise, to update θ_t to θ_{t+1} requires multiple update steps on the same batch of data. It would initialize $\theta = \theta_t$

, then repeatedly apply gradient descent (such as the Adam optimizer) to update θ until the surrogate advantage has stabilized. It would then assign θ_{t+1} to θ , and do it again. During this inner-loop, the first update to θ would not hit the $1 - \epsilon, 1 + \epsilon$ bounds, but as θ is updated further and further away from θ_t , it eventually starts hitting the bounds. For each such bound hit, the corresponding gradient becomes zero, and thus PPO avoid updating θ too far away from θ_t . This is important, because the surrogate loss assumes that the state-action pair s, a is sampled from what the agent would see if the agent runs the policy π_{θ_t} , but policy gradient should be on-policy. So, as θ changes, the surrogate loss becomes more and more off-policy. This is why keeping θ proximal to θ_t is necessary. If there is a reference policy π_{ref} that the trained policy should not diverge too far from, then additional KL divergence penalty can be added: $-\beta \mathbb{E}_{s, a \sim \pi_{\theta_t}} [\log \frac{\pi_{\theta}(a|s)}{\pi_{\text{ref}}(a|s)}]$ where β adjusts the strength of the penalty. This has been used in training reasoning language models with reinforcement learning from human feedback. The KL divergence penalty term can be estimated with lower variance using the equivalent form (see f-divergence for details): $-\beta \mathbb{E}_{s, a \sim \pi_{\theta_t}} [\log \frac{\pi_{\theta}(a|s)}{\pi_{\text{ref}}(a|s)} + \frac{\pi_{\text{ref}}(a|s)}{\pi_{\theta}(a|s)} - 1]$